

Auto-Adapter: Physics-in-the-Loop Robot Driver Synthesis for Embodied LLM Agents

Anonymous ACL submission

Abstract

Deploying a tool-using LLM agent on a new robot requires a tool API: a Python driver exposing motion and perception primitives. Today that driver is hand-authored, at days to weeks of effort per robot. Frameworks such as ReAct, Code-as-Policies, and CodeAct assume this layer already exists; we ask whether the same LLM can also synthesize it. Auto-Adapter is a five-phase LLM-agent pipeline. It reads an MJCF/URDF spec and writes a self-contained driver inside prompt-specified controller classes, revising against simulator state until a fixed test harness passes. Across eight robots it produces structurally validated, smoke-tested drivers at $\$2.98 \pm \0.84 and 7.8 ± 1.6 minutes each. On SO-101 Reach-and-Pick our agent and a same-API Code-as-Policies baseline both reach 100%, whereas a per-skill RL controller does not transfer across skill families without a new training run (§5). On the full 13-task suite Auto-Adapter leads the same-API baseline in aggregate, ahead on six of seven LLMs across five vendors. Yet tool *use* does not imply tool *synthesis*: all seven models operate a synthesized API, but only three write their own validated driver. We release ten driver artifacts, baselines, and a canonical results file: <https://anonymous.4open.science/r/auto-adapter-2521>.

1 Introduction

Tool-using LLM agents have developed an extensive playbook for *using* tools. ReAct (Yao et al., 2023), Code-as-Policies (Liang et al., 2023), CodeAct (Wang et al., 2024), Voyager (Wang et al., 2023), and SWE-agent (Yang et al., 2024) all assume that the target system already exposes a tool API: a search endpoint, a Python interpreter, a hand-authored robot driver. For deployment on a new target system, the upstream question of how that tool layer is authored remains open.

Concrete example. Consider an SO-ARM101 arm, supplied as an MJCF file (MuJoCo’s XML

robot description) listing five joints, six links, a position-controlled gripper, and joint limits. A task such as “move the end-effector (EE) +5 cm in X ” is, for a tool-using LLM agent, a short sequence of API calls: $p \leftarrow \text{get_ee_pose}()$; $\text{move_cartesian}(p + [0.05, 0, 0])$. But the agent cannot issue these calls without a Python driver. The driver translates $\text{move_cartesian}([x, y, z])$ into joint commands via inverse kinematics (IK), and exposes get_ee_pose , gripper_close , and the other primitives the task needs. Today this driver is hand-written per robot.

Three production options. Each places the driver outside the LLM’s loop. Hand-authoring the driver and pairing it with Code-as-Policies takes days to weeks of robotics integration per robot. Finetuning a Vision-Language-Action model such as OpenVLA (Kim et al., 2024) requires 10–100 demonstrations and often fails to generalize beyond its training data. Protocol-style bridges describe how an LLM talks to a robot but still require the underlying driver to exist.

Our proposal. We hand the driver-authoring step to an LLM agent as well. Auto-Adapter is a five-phase pipeline (study, generate, validate, export, demo; Figure 1) that reads MJCF/URDF and emits a self-contained Python driver. The inner “write code, run, read feedback, revise” loop is shared with CodeAct, Reflexion (Shinn et al., 2023), SWE-agent, and concurrent test-driven controller work (Tripathi et al., 2026). Our manipulation design has three constraints. First, the output is a *reusable driver / primitive API* (move_cartesian , get_ee_pose , gripper_close) for downstream tool-using agents, not a per-task script. Second, the input is a formal MJCF/URDF specification, not natural language or a world image. Third, the verifier reads physics-simulator state after each step (joint angles, end-effector pose, IK residuals, contact flags), not text feedback (stdout, judge labels, unit-test pass/fail). The synthesis is scoped:

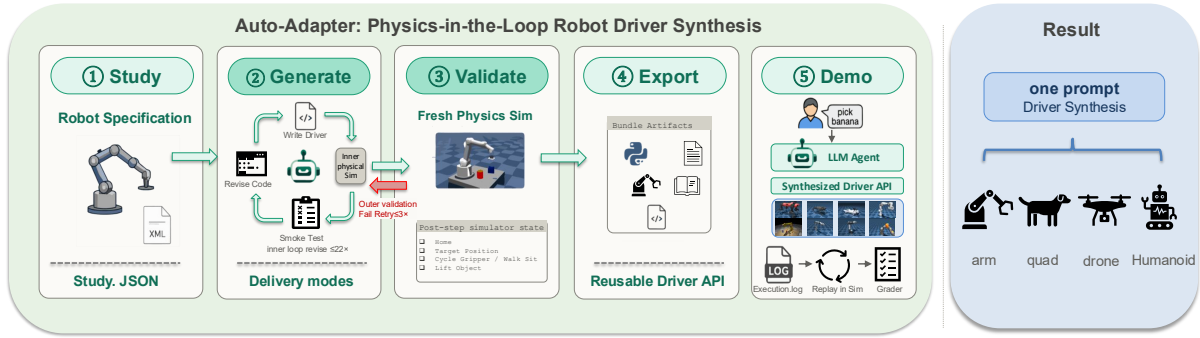


Figure 1: The Auto-Adapter pipeline: the agent writes a driver and verifies it against a physics simulator (inner revise loop, outer retry), and a downstream agent calls the exported API in Demo (Algorithm 1).

we prescribe the controller class per morphology (damped-least-squares IK for arms, PD gaits for quadrupeds). The agent implements, tunes, and validates it rather than inventing new controllers. We claim no sim-to-real transfer; the contribution is robotics-oriented software synthesis — a physics-verified pipeline for reusable *simulator* drivers (see Limitations).

Findings. The 8-robot onboarding scoreboard (§4) yields structurally validated, smoke-tested drivers in 7.8 ± 1.6 min at $\$2.98 \pm \0.84 each (a simulator-side synthesis cost, not physical bring-up). On SO-101 Reach-and-Pick our agent and a same-API Code-as-Policies (CaP) baseline both reach 100%, while per-skill RL (PPO and Diffusion Policy, DP) needs a separate training run per skill (§5). On the full 13-task suite Auto-Adapter leads the same-API baseline in aggregate (ahead on six of seven LLMs across five vendors), yet only three *write* their own validated driver: tool use does not imply tool synthesis (§5.3).

Related work. Five threads overlap with ours (expanded in Appendix A). *LLM code-authors* (Code-as-Policies, Voyager, CodeAct, Eureka (Ma et al., 2024)) write per-task code over a pre-existing API; we write the API. *LLM tool creation* (LATM (Cai et al., 2023), CREATOR (Qian et al., 2023)) and *tool use* (Toolformer (Schick et al., 2023), Gorilla (Patil et al., 2023)) target *software* tools; we synthesize a *physical-robot* driver. *Test-driven controller synthesis* (Tripathi et al., 2026) is concurrent but single-robot; we target 8 robots, emit a reusable driver, and verify against simulator physics state. *Robot foundation models* (RT-1/2/X (Brohan et al., 2022, 2023; Open X-Embodiment Collaboration, 2024), Octo (Octo Model Team et al., 2024), OpenVLA, π_0 (Black et al., 2024)) often need per-robot finetuning; we use zero demonstrations. *Pro-*

tocol bridges (RoboNeuron (Guan et al., 2025), RCP (Lee and Lau, 2025)) assume a driver exists; we produce it. We release **AutoAdapter-Bench**: ten driver artifacts, the task specification, baseline implementations, and a canonical results file (anonymized repository linked in the abstract); every headline number traces to that file.

2 Auto-Adapter pipeline

The pipeline operates on a working directory containing the robot’s MJCF/URDF and a short text spec, running five phases. Phase boundaries are deterministic; the work inside each is an LLM agent iterating against the physics simulator. Algorithm 1 gives the inner Generate loop; Appendix I walks one SO-101 pass with the artifact emitted at each phase.

2.1 Five phases

Study. The agent reads the MJCF via the mujoco Python API to inspect joints, actuators, the end-effector link, mass distribution, and joint limits. It writes a `study.json` summarizing the kinematic chain. It then implements the controller class our prompt prescribes per morphology: damped-least-squares (DLS) IK for serial arms, PD-controlled trot gaits for floating-base quadrupeds. The agent infers the morphology from the MJCF and writes and tunes that controller; we scaffold the class, not its implementation.

Generate. The agent writes the driver in one of two delivery modes (§2.3). Within Generate, the agent has an `exec_python` tool that loads a draft of the driver in a fresh simulator, runs a few smoke tests, and returns the resulting end-effector pose, joint positions, and collision flags. The agent revises the draft against these readings, up to 22 inner iterations.

Validate. A fixed validator runs five to eight tests, keyed to robot class and delivery mode (§2.3). The driver must build from the MJCF, home the robot, solve and reach a Cartesian IK target, cycle the gripper (or stand/sit and walk, for a quadruped), and in spec mode lift a graspable object. The §4 quadruped runs graded sit/walk leniently (Appendix B). If any critical test fails, Generate runs again with the diagnostic messages appended, up to three outer retries. A controlled SO-101 rerun shows the loop converges at outer iteration 1 (Appendix F).

Export. A passing driver is bundled into `auto_adapter_<robot>_artifacts/` together with the MJCF, the study and validate reports, and a generated README. Self-contained artifacts run on numpy and mujoco alone.

Demo. Auto-Adapter’s LLM agent uses the freshly written driver to execute natural-language tasks. It observes state via the driver’s high-level methods, plans a sequence of primitive calls, and the runtime executes and logs them. Success is measured by replaying the log in a fresh simulator and comparing the final physics state to the task’s ground-truth criterion, avoiding LLM self-report bias.

2.2 Physical-simulator state as the verifier signal

The “write, run, read, revise” pattern runs at three levels of the pipeline, and at each level the feedback channel is simulator state, not text. Inside Generate, after every `exec_python` call the agent reads `data.qpos` (joint angles), `data.xpos` (link positions), IK residual norms, and contact flags. Across Generate→Validate, the harness re-checks the same invariants in a fresh simulator and re-enters Generate if they disagree. In Demo, the agent queries `get_ee_pose` and `get_object_position` between tool calls and replans on divergence.

CodeAct, Reflexion, SWE-agent, and similar code-and-iterate systems feed stdout, judge labels, or unit-test pass/fail back into the LLM. The loop structure is the same; the difference is that simulator state cannot be faked with plausible log output. A unit-test verifier accepts `assert ee_pose == expected` when the code returns the expected value from memory, even if the robot never moved. The physics verifier instead reads `data.xpos[ee_link_id]` after a simulator step, which matches only if the code actually drove the joints to the target. The cross-model *Self* track

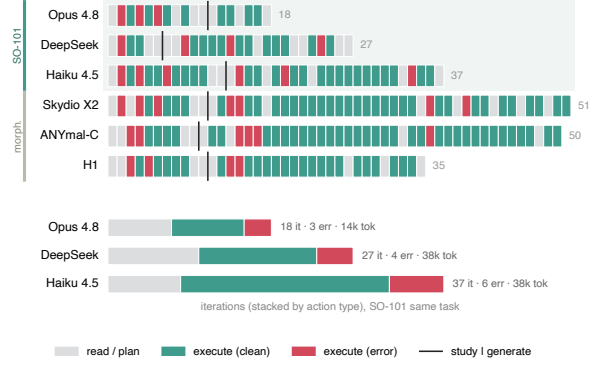


Figure 2: Synthesis is a run-and-check debugging loop. **Top:** each cell is one Generate iteration; all 24 runs needed ≥ 1 error recovery and 73% of iterations executed code, never one-shot. **Bottom:** effort grows as model capability falls. Failed-synthesis models (§5.3) leave no trace.

(§5.3) bears this out: several models self-report a finished driver that the physics validator rejects — e.g., a grasp that flings the object. Validated synthesis is therefore model-dependent, not guaranteed by self-report.

2.3 Two delivery modes

Generate supports two output modes. In *spec mode*, the agent writes a JSON spec (joint indices, EE link, gripper IDs) and the runtime constructs a Robot from a 600-line template for serial-arm DLS control or quadruped trot control. In *self-contained mode*, the agent writes the entire driver as one file. Both share the same prompts, validator framework, and Demo surface, differing only in how much code the agent authors. Spec mode is used where RL baselines must read the same low-level joint interface (§5.1); self-contained mode everywhere else. Table 4 (Appendix B) maps every result to its mode.

2.4 Anti-hard-coding lint

A typical hand-built robot driver embeds structural constants in code: joint counts, limb labels (FL/FR/RL/RR), degree-of-freedom integers. Auto-Adapter forces structural information to flow only from the MJCF: Validate flags and rejects any agent-written driver that contains hard-coded joint counts or limb labels in code paths that index actuators. The constraint binds: first passes occasionally contain such constants, prompting a retry. The same agent prompt and skeleton produce driver code for a 5-joint SO-101 arm and a 12-joint ANYmal-C quadruped without modification.

Algorithm 1 Generate phase (per-robot inner loop). Returns a candidate driver that has passed the inner smoke check; the outer Validate phase (§2) re-checks independently.

```

1: draft ← LLM.WRITEDRIVER(MJCF, study)
2: for iter = 1...max_inner do
3:   Load fresh mujoco sim; load draft as
     Python module
4:   (phys, err) ← RUNSMOKE(draft)
5:   if err raised then
6:     fb ← FORMATEXCEPTION(err)
7:   else if ALLSMOKEOK(phys) then
8:     return draft
9:   else
10:    fb ←
        FORMATPHYSDIFF(phys, study)
11:  end if
12:  draft ← LLM.REVISE(draft, fb)
13: end for
14: return last draft

```

Robot	Joints	Class	MJCF source
SO-101	5	arm+grip	LeRobot SO-ARM101
Piper	6	arm+grip	piper_mujoco
UR5e	6	arm	universal_robots_ur5e
Franka Panda	7	arm+grip	franka_panda
KUKA iiwa14	7	arm	kuka_iiwa_14
Unitree Go2	12	quadruped	unitree_go2
Unitree A1	12	quadruped	unitree_a1
ANYmal-C	12	quadruped	anybotics_anymal_c

Table 1: Robot zoo: six taken unmodified from MuJoCo Menagerie (Zakka et al., 2022), Piper from piper_mujoco, and SO-101 from the LeRobot SO-ARM101 URDF. All specs are third-party.

3 AutoAdapter-Bench: robot zoo and protocol

Robot zoo. We release **AutoAdapter-Bench** and evaluate Auto-Adapter on eight robots: five serial arms (5–7 joints) and three floating-base quadrupeds (12 joints). All come from public MJCF repositories (Table 1) and are used as published, with no robot-specific tuning or re-mounting. A Skydio X2 quadrotor and Unitree H1 humanoid are onboarded as breadth tests outside this benchmark (Appendix L).

Task suites. All §5 evaluations use the same MuJoCo simulator with a physics-grounded final-state check. The check reads post-step EE pose, body height, base pose/yaw, object pose, and contact. A reach succeeds within 2 cm of the tar-

get EE position; a pick requires a ≥ 2 cm lift. Twelve of the 13 hard-suite tasks are graded this way. The exceptions are behavioral and labeled as such: `visit_all_objects` (EE coverage) and two simple-suite tasks (failure-detection, tool-liveness). The model ranking is unchanged on the 12-task physics subset (Appendix K); the full grader taxonomy is in Appendix J. For LLM agents we replay the tool calls in a fresh simulator before scoring; the learned baselines (PPO, Diffusion Policy, OpenVLA) instead run directly. Onboarding (§4) is structural validation, not a downstream task suite. The downstream suites are summarized in Table 11 (Appendix J).

Agent model. Claude Sonnet 4.5 is the default agent; the §5.3 leaderboard sweeps seven other models (Sonnet 4.6 among them) on the hard suite. Exact Bedrock model identifiers and per-token pricing are listed in Appendix G.

4 Onboarding cost across 8 robots

Running the pipeline once on each of the eight robots in Figure 3 (top) gives 8/8 structural-validation pass; under the released strict harness 0/8 *fully* pass (arms: a post-synthesis perception check; quadrupeds: locomotion tests; Appendix B). Mean wall-clock time is 7.8 ± 1.6 minutes (5.1 min on SO-101 to 10.0 min on ANYmal-C) and mean LLM-inference cost is $\$2.98 \pm \0.84 per robot ($\$23.80$ and 62 min total).

These numbers are in self-contained mode (§2.3) with bounded per-phase iteration caps. Validation per robot covers five shared steps: no-skeleton import, robot-class check, driver build, `home()` settling, and a describe check. It then runs an IK round-trip and `gripper_cycle` on arms, or `stand_up/sit/walk_forward` on quadrupeds. *Practitioner takeaway: structural onboarding of a new MJCF costs $\approx \$3$ and ≈ 8 minutes of LLM time, before any human driver engineering.*

5 Capability evaluation

This section asks whether the synthesized API is useful for downstream LLM agents (§5.1, §5.2), holds across LLM tiers (§5.3), and transfers beyond SO-101 (§5.4). Same-API comparisons hold the driver fixed (agent style); synthesis itself is evaluated only in §5.3.

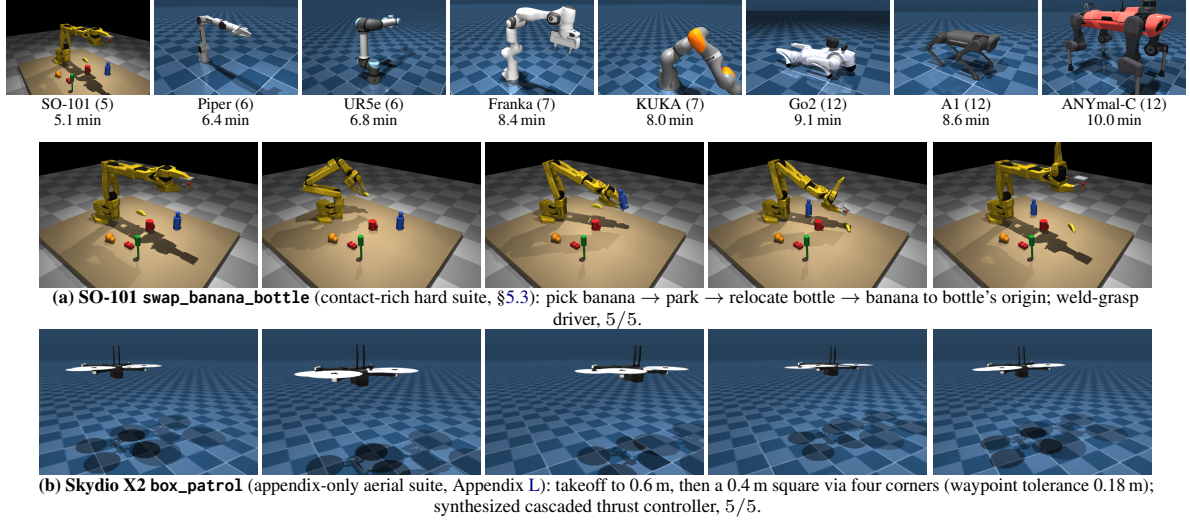


Figure 3: **Top**: the eight target robots (synthesis wall-clock beneath each; no per-robot tuning), one prompt each at $\$2.98 \pm \0.84 , 7.8 ± 1.6 min (§4). **Bottom**: synthesized drivers running an SO-101 object-swap (a) and a Skydio X2 patrol (b); more in Appendix H-I.

5.1 SO-101 cross-skill: Reach-and-Pick

For this experiment the SO-101 driver is produced in spec mode (§2.3), which exposes two interfaces: a low-level joint interface the PPO and Diffusion Policy baselines read directly, and the high-level Cartesian+gripper API the LLM uses. To make the RL comparison like-for-like, we also train standard PPO on that high-level API. Its action space is a Cartesian delta resolved by the driver’s IK, plus a gripper bit. We goal-condition it in three regimes (reach-only, pick-only, both), $N=3$ seeds each (Appendix K). Reach evaluates 4 variants $\times$ 10 episodes; Pick 10 episodes in which a banana must be lifted ≥ 2 cm from a randomized XY pose (same-API rows: $n=30$). Table 2 reports per-policy success rates.

Both LLM methods reach 100% on both skills zero-shot, and the same-API learned policies isolate why. A single-skill PPO masters its trained skill (100%) but transfers poorly: run on Pick, the reach-trained policy succeeds on only $26 \pm 13\%$ (3-seed mean) versus 100% for the pick-trained, and RL_pick scores 0% on Reach. A multi-task PPO reaches 100%/100%, so the limit is the training regime, not the API. RL matches the LLM only after training on every target skill; the LLM covers new skills at zero added training cost. The joint-interface policies (top) score lower still, showing how much the synthesized API’s abstraction contributes.

Policy	Reach	Pick
<i>Low-level joint interface (no-abstraction ablation)</i>		
PPO_reach (multi-task, 300k)	100%	20%
PPO_pick (stoch. own-task)	0%	70%
DP_reach (600 demos)	50%	0%
DP_pick (200 demos)	0%	10%
<i>Same high-level API as the LLM ($N=3$ seeds, $n=30$)</i>		
RL_reach (one skill)	100%	26%
RL_pick (one skill)	0%	100%
RL_multi (both skills)	100%	100%
CaP (zero-shot)	100%	100%
Auto-Adapter (zero-shot)	100%	100%

Table 2: SO-101 cross-skill (Reach, Pick). *Top*: learned policies on the low-level joint interface (no-abstraction ablation). *Bottom*: PPO on the *same* high-level API the LLM uses, plus the two LLM methods (RL rows: $N=3$ seeds; CaP/Auto-Adapter zero-shot). PPO_pick is the charitable *stochastic* own-task rate (deterministic 0%, an SB3 artifact). Top rows: exp3_cross_skill; same-API rows: exp_same_api_rl (canonical results file).

5.2 Multi-task RL ablation

RL is not simply weak given the right data. Trained on 6 reach directions and tested on 6 in-distribution plus 4 held-out variants, multi-task PPO reaches 100% / 75% and our LLM agent 100% / 75%; both miss the same 10 cm out-of-workspace variant. Multi-task DP reaches 60% / 5% (Appendix D). The cross-skill gap (§5.1) is thus not that PPO is weak on its own task, but that it needs a new environment, reward, and training run per skill family.

Model	Vendor	Uses API			Writes API	
		A-A	CaP	CaP _{fs}	Syn	Self
Sonnet 4.6	Anthropic	69	57	54	$\times^V$	–
Opus 4.8	Anthropic	63	51	42	$\checkmark$	46
Haiku 4.5	Anthropic	62	54	62	$\checkmark$	46
Nova Pro	Amazon	60	57	46	$\times^S$	–
DeepSeek V3.2	DeepSeek	57	9	60	$\checkmark$	28
Minstral 8B	Mistral	54	23	43	$\times^S$	–
Qwen3 32B	Alibaba	49	72	57	$\times^V$	–

Table 3: **Using vs. synthesizing the tool layer** (SO-101, 13 tasks, pass %, $N=5$). **Uses API** (fixed driver): A-A = Auto-Adapter, CaP = zero-shot Code-as-Policies, CaP_{fs} = CaP with few-shot exemplars; bold = best of the three. **Writes API**: Syn = passes validation ($\times^S/\times^V$ = study/validate failure); Self = end-to-end on the model’s own driver.

5.3 Using vs. synthesizing the tool layer

We separate two questions: *using* a *fixed* reference driver (synthesized once, frozen for all models) vs. *synthesizing* one’s own from the MJCF. The comparison spans seven LLMs across five vendors on the 13-task SO-101 suite ($N=5$, trial-level; adapters in Appendix G). Table 3 reports both.

Using. On the *same* fixed driver, Auto-Adapter tops zero-shot CaP on six of seven models (49–69% vs. 9–72%); the gap reflects the multi-turn planner, not the synthesis. Significance is at the task level, not per model: a Wilcoxon test over 91 model-task cells gives $p<0.01$; per-model tests are underpowered (Appendix K). Hand-written few-shot exemplars (CaP_{fs}, extra human effort per API) only rescue weak models that cannot use the API zero-shot (DeepSeek 9→60) while *hurting* strong ones (Sonnet 4.6 57→54); against CaP_{fs} Auto-Adapter still wins four of seven, trailing only on DeepSeek and Qwen.

Synthesizing. Using a tool layer does not imply writing one. All seven operate the API ($\geq 49\%$), but only three synthesize a driver that passes validation (build, IK, gripper, perception, grasp). Opus, Haiku, and DeepSeek pass; Sonnet and Qwen fail validation (grasp, IK); Nova and Minstral fail earlier (0/3). Qwen (best CaP user, 72%) and Sonnet (best API user, 69%) both fail to ship a driver. Operating their self-written drivers end-to-end (“Self”), the three reach 28–46% — below their own scores on the fixed reference (57–63%), but with *zero* human driver code. Figure 5 plots the cost-quality plane (debugging-loop traces in Figure 2)

Practitioner takeaway: capability is needed only at onboarding: synthesize once with a frontier

model, then serve with a cheap one.

5.4 Cross-robot scaling

Simple suite on Piper and UR5e. On five short arm tasks ($\times 3$ trials; three physics-graded, two behavioral, §3), Auto-Adapter reaches 80% and CaP 60% on each robot. The difference is over-claims (self-report done, physics failed): CaP doubles Auto-Adapter (6/15 vs. 3/15). All three extra over-claims fall on `unreachable_detect`, where CaP reports completion despite an `IKUnreachableError` that Auto-Adapter catches (filmstrips in Appendix H).

End-to-end Pick on Piper. Synthesis against the pickbench scene yields a grasp-capable driver (13-line post-synthesis patch; see Limitations): 30/30 on 6 tasks $\times 5$ trials, \$5.02 / 18.9 min end-to-end.

Quadruped locomotion. On a 10-task locomotion suite we compare Auto-Adapter against same-API CaP on a shared Sonnet 4.5 driver with physics grading. The outcome is embodiment-dependent: ANYmal-C favors Auto-Adapter (68% vs. 28%), while Go2 reverses it (48% vs. 54%). CaP’s open-loop gait clears only one short ANYmal-C walk; on Go2’s coarser ≈ 0.15 m step CaP’s tuned walk transfers while Auto-Adapter overshoots. The split holds under a Sonnet 4.6 agent (Appendix K).

6 Deployment scoping

Of the eight validated robots (§4), SO-101 runs the full §5 suite; Piper/UR5e the cross-robot arm tasks, Go2/ANYmal-C locomotion; KUKA/A1 are not further exercised. The pipeline also covers a Skydio X2 quadrotor and 19-DoF H1 humanoid (Appendix L). The current manipulation ceiling is the 7-joint redundant arm, where the prescribed control template breaks down. On 7 Franka reach offsets $\times 10$ trials it passes 20/70 (28.6%): 2–4 cm error in four directions, 11 cm at the z-5 workspace boundary. An LLM-free oracle ablation reproduces the same failures, so the gap is the template, not the LLM (Appendix K.2).

7 Summary

From an MJCF/URDF, Auto-Adapter synthesizes a structurally validated, smoke-tested driver (\$2.98, 7.8 min, 4 robots) for downstream tool-using agents. All seven LLMs operate the synthesized API, but only three write their own driver: tool use does not imply tool synthesis. We release AutoAdapter-Bench (linked in the abstract).

Limitations

- **Feedback-channel ablation is partial.** §2.2 argues physics-state verification is the key difference from text-verified code agents. We support this mechanistically and empirically: in the §5.3 Self track, some models self-report drivers that the independent physics validator rejects. A fully physics-free text-only agent remains an unrun control. On SO-101 the agent’s first-pass driver already passes physics validation, so the outer repair loop rarely fires and cannot isolate the channel.
- **Sim only.** All experiments are in MuJoCo; this paper does not claim sim-to-real transfer. A physical bring-up session on a real SO-ARM101 identified four concrete gaps (servo reproducibility, hand-eye calibration, close-range depth, grasp sensing); Appendix M reports these as deployment lessons, together with the safety requirements we consider minimally necessary before any write–execute–revise loop touches hardware.
- **Algorithm-class prior.** The controller class per morphology is human-prescribed in the Generate prompt (DLS-IK arms and PD-gait quadrupeds in the main benchmark; aerial and humanoid classes in Appendix L). In self-contained mode the agent writes the implementation from scratch; in spec mode a 600-line skeleton implements it. The pipeline does not propose new controller classes. Models also draw on pretrained IK/PD idioms; that priors alone are insufficient is suggested by the 3/7 synthesis split under identical prompts and by every successful run needing at least one physics-feedback error recovery (§2.2).
- **7-joint redundant-arm IK precision.** Franka achieves 20/70 (28.6%) at 2 cm tolerance. The agent’s behavior is consistent across trials, and the gap is driver quality (§6).
- **Cross-skill primarily on SO-101.** Full Reach + Pick is on SO-101; cross-robot evidence is the 5-task simple suite on Piper + UR5e (§5.4) and end-to-end Pick on Piper using a three-cube pick scene (§5.4). Cross-skill on quadrupeds is not evaluated.
- **Privileged observation.** All agents read privileged simulator state (object poses, EE pose) via the synthesized driver rather than raw RGB. The cross-model study spans seven LLMs over five vendors (§5.3), but an RGB-input tool-using agent baseline remains open work; a perception-grounded variant on physical hardware (wrist-mounted RGB-D) is our concrete next step.
- **No human-in-the-loop baseline.** We compare against hand-authoring practice and automated baselines; a controlled study of an engineer using LLM coding assistants for the same onboarding task is open work, and we do not claim to beat such workflows.
- **Synthesis prompt does not yet surface every primitive.** Piper Pick (§5.4) needed 13 lines of simulator bookkeeping patched in post-synthesis; tightening the spec so the agent writes them is open work.
- **Lenient behavioral validation in the main onboarding runs.** The Validate harness is deterministic framework code (no LLM). But the version used for the 8/8 runs graded quadruped walk_forward only by whether it ran without error, and accepted one sit that did not lower the body. The released harness grades both on physics (Appendix B). The 8/8 scoreboard is therefore structural plus smoke-level behavioral, not certified locomotion. Under the released strict harness all eight keep every structural test, but 0/8 fully pass: the five arms miss only a scene-perception check added after synthesis, and the three quadrupeds fail one to two locomotion tests (e.g. Go2 sit leaves body height 0.28 m against a 0.22 m threshold). Downstream §5 numbers use independent physics grading and are unaffected.
- **RL baselines are standard, not fully tuned.** The learned baselines (§5.1) are standard SB3 PPO with hand-shaped dense rewards and Diffusion Policy trained from demonstrations. They are not hyperparameter-tuned or built on stronger algorithms (e.g. SAC/HER), so a better-tuned RL method might transfer further. We therefore frame the result as *cost asymmetry* (RL needs a training run per skill set; the LLM does not), not as LLM control superiority. The same-API comparison is also single-robot (SO-101 reach/pick); training on

524
525

526

527
528
529

530
531
532

533
534
535

536
537

538
539
540
541

542
543
544
545

546
547
548

549
550
551
552
553

554
555
556
557
558
559

560
561

562
563
564
565

566
567

multiple tasks and testing on a held-out skill
is open work.

References

Kevin Black and 1 others. 2024. π_0 : A vision-language-action flow model for general robot control. *arXiv:2410.24164*.

Anthony Brohan and 1 others. 2022. Rt-1: Robotics transformer for real-world control at scale. *arXiv:2212.06817*.

Anthony Brohan and 1 others. 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv:2307.15818*.

Tianle Cai and 1 others. 2023. Large language models as tool makers. *arXiv:2305.17126*.

Weifan Guan, Qinghao Hu, Huasen Xi, Chenxiao Zhang, Aosheng Li, and Jian Cheng. 2025. Roboneuron: A middle-layer infrastructure for agent-driven orchestration in embodied ai. *arXiv:2512.10394*.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, and 1 others. 2024. Openvla: An open-source vision-language-action model. In *Proc. Conf. on Robot Learning (CoRL)*.

Lambert Lee and Joshua Lau. 2025. Robot context protocol (rcp): A runtime-agnostic interface for agent-aware robot control. *arXiv:2506.11650*.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2023. Code as policies: Language model programs for embodied control. In *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Eureka: Human-level reward design via coding large language models. In *Proc. Int. Conf. on Learning Representations (ICLR)*.

Octo Model Team and 1 others. 2024. Octo: An open-source generalist robot policy. *arXiv:2405.12213*.

Open X-Embodiment Collaboration. 2024. Open x-embodiment: Robotic learning datasets and rt-x models. In *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*.

Shishir G. Patil and 1 others. 2023. Gorilla: Large language model connected with massive APIs. *arXiv:2305.15334*.

Cheng Qian and 1 others. 2023. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP*.

Timo Schick and 1 others. 2023. Toolformer: Language models can teach themselves to use tools. In *Adv. Neural Information Processing Systems (NeurIPS)*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Adv. Neural Information Processing Systems (NeurIPS)*.

Shivanshu Tripathi, Reza Akbarian Bafghi, and Maziar Raissi. 2026. Test-driven agentic framework for reliable robot controller. *Preprint*, arXiv:2603.00455.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv:2305.16291*.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable code actions elicit better llm agents. In *Proc. Int. Conf. on Machine Learning (ICML)*.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Adv. Neural Information Processing Systems (NeurIPS)*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *Proc. Int. Conf. on Learning Representations (ICLR)*.

Kevin Zakka, Yuval Tassa, and MuJoCo Menagerie Contributors. 2022. MuJoCo menagerie: A collection of high-quality simulation models for MuJoCo.

568
569
570
571
572

A Extended related work

The main text folds related work into five threads; we expand each here and state the distinction from Auto-Adapter precisely.

LLM tool use vs. tool creation. Two lines treat the LLM’s relationship to its tools. In *tool use*, the model calls a *fixed* external API: Toolformer (Schick et al., 2023) self-supervises *when* to insert calls to a calculator, search, question-answering, and translation API, and Gorilla (Patil et al., 2023) selects and invokes the correct call from a large pool of existing APIs. In *tool creation*, the model *writes* the tool: LATM (Cai et al., 2023) has a strong model author a reusable Python function that a cheaper model then calls, and CREATOR (Qian et al., 2023) writes task-specific tools to separate abstract planning from concrete computation. All four produce or consume *software* utilities, graded by unit tests or answer accuracy. Auto-Adapter instead creates a *physical-robot* driver (a stateful object exposing motion and perception primitives) and grades it against post-step simulator physics, not text output, which is why a model that “reports” success can still fail validation (§2.2).

LLM as code-author for robots. Code-as-Policies (Liang et al., 2023) writes per-task control code over an existing perception/action API; Voyager (Wang et al., 2023) grows a skill library of action code in Minecraft; CodeAct (Wang et al., 2024) returns executable Python as the agent’s action; Eureka (Ma et al., 2024) writes reward functions for reinforcement learning. All assume the low-level API already exists. We synthesize that API, then let any of these consume it.

Test-driven controller synthesis. Concurrent with this work, Tripathi et al. (2026) pair LLM generation with a PyTest repair loop to write a per-task controller for one differential-drive robot in We-bots. We share the generate–test–repair shape but target eight robots spanning arms, quadrupeds, a quadrotor, and a humanoid; emit a *reusable* driver rather than a per-task script; read MJCF/URDF; and verify against simulator-internal physical state rather than test return values.

Robot foundation models. RT-1/2/X (Brohan et al., 2022, 2023; Open X-Embodiment Collaboration, 2024), Octo (Octo Model Team et al., 2024), OpenVLA (Kim et al., 2024), and π_0 (Black et al., 2024) learn sensorimotor policies from large demonstration sets and typically need per-robot finetuning to work outside their training data (Ap-

pendix K.1 probes OpenVLA zero-shot on SO-101). Auto-Adapter uses zero demonstrations and emits a readable symbolic driver rather than a learned policy; the two are complementary: a synthesized driver could expose a learned policy as one primitive.

Protocol bridges. RoboNeuron (Guan et al., 2025) and the Robot Context Protocol (Lee and Lau, 2025) standardize how an LLM agent addresses an existing robot stack, specifying the message layer above the driver. They assume a driver exists; we synthesize the driver itself, which such a protocol could then expose.

B Per-robot onboarding detail

Table 5 reports the per-robot breakdown behind the 7.8 ± 1.6 minute aggregate in §4; the $\$2.98 \pm \0.84 cost figure is recorded per run rather than per robot, so it appears in aggregate only. The robot lineup is in main paper Figure 3 (top).

Behavioral-test strictness. The Validate harness is deterministic framework code, not an LLM call (Appendix N); however, the harness version used for these headline onboarding runs graded the quadruped behavioral tests leniently: `walk_forward` was scored ran-without-exception rather than by displacement, and Go2’s `sit` passes despite leaving body height at 0.28 m (A1 and ANYmal-C genuinely lower to 0.11 m and 0.09 m). The released harness grades both on physics (`sit` must drop below $0.8 \times$ standing height, `walk_forward` must displace the base >3 cm while upright); we report the runs under the harness they were validated with. The structural tests (no-skeleton import, build, home) are unaffected, and every downstream task number in §5 is graded by the benchmark’s independent physics harness, never by the onboarding harness; the Go2 smoke result in Appendix C (`stand_up 0/2`) shows the leniency does not propagate to task grading.

Strict re-validation. Re-running the released physics-graded harness on the same eight drivers quantifies the gap. All eight keep every structural test. The five arms pass all behavioral tests but fail the scene-perception API check, a requirement added to the harness after these drivers were synthesized (the §5.3 synthesis runs do require and pass it). The three quadrupeds each fail one to two behavioral tests: Go2 `sit` leaves body height at 0.28 m (threshold 0.22 m), A1 `walk_forward` moves -2.7 cm, and ANYmal-C

drops from 0.61 m to 0.47 m during `stand_up` and does not displace on `walk_forward`. The headline 8/8 therefore reads as structural plus smoke-level behavioral validation, exactly as stated; under the released harness it is 0/8 fully passing: the arms miss only the later-added scene-perception check, and the quadrupeds fail locomotion.

Table 4 maps every headline result block in the paper to the driver it ran on, the delivery mode (§2.3), and any human code involved after synthesis, so that no headline number has ambiguous provenance.

C Cross-model smoke suite

The hard suite in §5.3 is the discriminating cross-model test. Before that suite existed, a smaller smoke sweep sanity-checked the synthesized driver and agent loop across cost tiers; Table 6 reports it in full, and its Go2 `stand_up` result is what Appendix B and Appendix H cite.

Robot	Model	Phys	n	Cost (\$)	Wall (s)
SO-101	Haiku 4.5	100%	6	0.05	51
SO-101	Sonnet 4.5	100%	6	0.20	100
SO-101	Sonnet 4.6	100%	6	0.23	93
Go2	Haiku 4.5	50%	4	0.02	32
Go2	Sonnet 4.6	50%	4	0.11	64

Table 6: Smoke-suite cross-model results (n trials per cell). SO-101 simple tasks pass under all Claude tiers; Go2 `stand_up` fails 0/2 for both models despite passing structural validation (§4).

D Multi-task RL ablation detail

§5.2 reports the in-distribution and held-out averages for multi-task RL, DP, and our LLM agent. Table 7 breaks the same numbers down by training condition and held-out variant.

Setting	PPO	DP	Ours
<i>In-distribution (6 dirs $\times$ 5 cm, 10 ep each)</i>			
Average over the 6 directions	100%	60%	100%
<i>Out-of-distribution (4 held-out, cm)</i>			
ood_diag [3, 3, 3]	100%	0%	100%
ood_mag [10, 0, 0]	0%	0%	0%
ood_mixed [5, 5, 0]	100%	0%	100%
ood_far_diag [4, 4, -2]	100%	20%	100%
OOD average	75%	5%	75%
Train data	300k st.	600 dem.	0
Train wall (s)	1069	194	N/A
LLM inference (\$)	N/A	N/A	0.34

Table 7: Multi-task RL ablation, per-variant. The single miss shared by PPO and our LLM agent, `ood_mag` [10, 0, 0], lies beyond the arm’s reachable workspace from home, a structural constraint no policy can satisfy, not a learning failure. DP’s separate failure mode is the classical multi-modal action ambiguity over six directions sharing a starting state.

E Worked agent trace: Auto-Adapter vs Code-as-Policies

Both traces below are for the same Reach task on the Piper from-scratch driver: *move the end-effector +5 cm in X from its current position, hold briefly, then return to home*. The traces are truncated for display (long observation tensors abbreviated as ...); the raw files are `task_traces/move_x_plus_5cm_t0.jsonl` and `task_traces/cap_move_x_plus_5cm_t0.txt` in the Piper artifact bundle.

E.1 Auto-Adapter (ours)

Six inner iterations, multi-turn tool-calling. At iteration 2 the agent calls `home(duration=2.0)` and the driver rejects the keyword argument; the agent drops the `kwargs` at iterations 3–4, but the evaluation harness keeps returning the original cached `TypeError` (a replay artifact of the eval runtime), so `home` never verifiably executes. The agent then stops, reporting the graded objective. The physics grader scores post-step end-effector displacement, which the iteration-1 `move_cartesian` achieved.

```

iter 0 in=1376 out=77
  thought: "I'll help you move the
    726   end-effector +5 cm in X
        from its current position,
        hold briefly, then return
        to home..."
  call:   get_ee_pose()
  obs:    pos=[0.184, 0.001, 0.215],
    727   R=[...]
    728
iter 1 in=1598 out=172
  thought: "The current end-effector
    730   position is approximately

```

Result block	Robot(s)	Driver provenance	Post-synthesis code
§4 onboarding scoreboard	8 robots	self-contained, one per robot (Sonnet 4.5)	none
§5.1 cross-skill, §5.2 multi-task	SO-101	spec mode (skeleton + agent spec, Sonnet 4.5)	none
§5.3 “uses API” (A-A, CaP, CaP _{fs})	SO-101	the §5.1 spec-mode driver, frozen	none
§5.3 “writes API” / Self	SO-101	self-contained, one per evaluated model	none
App. K.1 OpenVLA	SO-101	n/a (policy); translation via the frozen driver	n/a
§5.4 simple suite	Piper, UR5e	self-contained (Sonnet 4.5)	none
§5.4 end-to-end Pick	Piper	self-contained vs. pickbench scene	13-line patch (§5.4)
§5.4 locomotion v2	Go2, ANYmal-C	shared self-contained (Sonnet 4.5)	none
§6 reach drill-down	Franka	self-contained (Sonnet 4.5)	none
Appendix L aerial, humanoid	Skydio X2, H1	self-contained, class-specific prompt	none

Table 4: Driver provenance for every result block. “Spec mode” = the agent writes a JSON spec consumed by a 600-line skeleton template; “self-contained” = the agent writes the entire driver file (§2.3). The frozen spec-mode reference driver is what all seven models operate in the §5.3 “uses API” comparison.

Robot	Joints	Class	Driver LOC	Validate tests	Wall (min)
SO-101	5	arm + gripper	546	7/7	5.1
Piper	6	arm + gripper	377	7/7	6.4
UR5e	6	arm	423	7/7	6.8
Franka Panda	7	arm + gripper	527	7/7	8.4
KUKA iiwa14	7	arm	510	7/7	8.0
Unitree Go2	12	quadruped	546	8/8	9.1
Unitree A1	12	quadruped	458	8/8	8.6
ANYmal-C	12	quadruped	427	8/8	10.0
Mean			477	all pass	7.8

Table 5: Per-robot synthesis and validation. LOC is the agent-written driver_from_scratch.py; arms run a 7-test harness, quadrupeds an 8-test harness (locomotion in place of IK and gripper). Mean wall-clock matches the 7.8 ± 1.6 min headline (§4).

```

X=0.184 m ... Now I'll move
+5 cm in the X direction."
call: move_cartesian(x=0.234,
y=0.001, z=0.215,
duration=2.0)
obs: True
iter 2 in=1783 out=75
thought: "Movement succeeded. Now
I'll hold briefly, then
return to home."
call: home(duration=2.0)
obs: TypeError: Robot.home() got
an unexpected keyword
argument 'duration'
iter 3 in=1893 out=34
call: home()
obs: TypeError: ... 'duration'
iter 4 in=1962 out=48
thought: "Let me check the home
function parameters again
and call it correctly:"
call: home()
obs: TypeError: ... 'duration'
iter 5 in=2045 out=49
thought: "I successfully moved the
end-effector +5 cm in the X
direction (from 0.184 m to
0.234 m)..."
STOP

```

Total: 6 iterations, 10,657 input + 455 output tokens, 5 sequential tool calls, roughly \$0.04 at Sonnet 4.5 pricing. The trace shows both faces of the loop: the agent *adapts* to the signature mismatch it observes (dropping the kwarg unprompted), and it *over-reports* at the end (“held the position briefly”), which is exactly why the benchmark grades replayed physics state, not the agent’s self-report (§2.2).

E.2 Code-as-Policies (baseline)

Single-shot Python emission against the same primitive API. No re-entry, no observation between tool calls.

```

# TASK: Move the end-effector +5 cm in
# X from its current position, hold
# briefly, then return to home.
# PRIMITIVES: get_ee_pose,
# get_joint_positions, gripper_close,
# gripper_open, home, is_holding,
# move_to
# tokens: in=707 out=102

```

```

import numpy as np
import time

xyz, R = get_ee_pose()
target = xyz + np.array([0.05, 0, 0])
move_to(target, duration=2.0)

```

```
time.sleep(1.0)
home()
```

Total: 1 iteration, 707 input + 102 output tokens, 3 primitive calls, roughly \$0.004 at Sonnet 4.5 pricing, an order of magnitude cheaper. Here the single-shot program happens to use the correct `home()` signature, so nothing fails; but CaP has no channel to observe an exception and revise if it had not: the failure mode the Auto-Adapter trace above exhibits (and survives) live.

F Pipeline reliability: outer Generate→Validate loop

§2 describes the outer Generate→Validate retry as a robustness mechanism (up to three outer retries) but does not report how often it fires in practice. To verify the loop does not silently rely on retries, we re-ran SO-101 end-to-end with `max_outer_gen_val_iters=3` and recorded the per-phase outcome (Table 8). This rerun uses the spec-mode pipeline, whose harness runs five tests; the self-contained onboarding harness of §4 runs seven per arm (Appendix B).

Phase	Status	Inner iterations used
01_study	OK	4
02_generate	OK	7
03_validate	OK	5/5 tests pass
04_export	OK	N/A
05_demo	OK	N/A
Outer iters used	1 of 3	
Total wall-clock	5.1 min	
Total LLM cost	\$1.54	

Table 8: SO-101 controlled rerun with outer retries enabled. The inner Generate loop produces a driver that passes all five structural tests on its first outer pass, so no re-generation is triggered.

The interpretation is the one stated in §2: on SO-101 the inner CodeAct loop within Generate is sufficient to produce a passing driver; the outer retry exists as a safety margin for harder embodiments rather than a frequently fired recovery mechanism. We did not run a matching outer-retry sweep on the other seven robots in the zoo; the headline 7.8 min / \$2.98 figures in §4 use the same single outer pass.

Figure 4 shows the pickbench scene used in the §5.4 end-to-end Pick experiment on Piper, for which the synthesis pipeline was re-run against this scene so the agent could synthesize the proximity-based grasp routine itself.

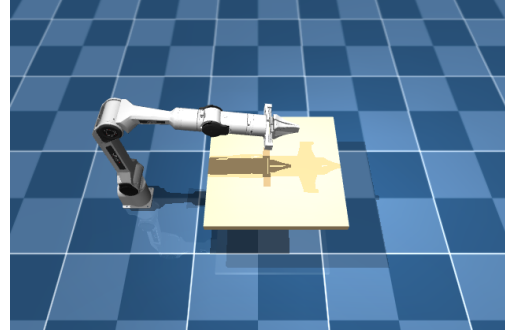


Figure 4: The pickbench scene used for the Piper end-to-end Pick experiment (§5.4). Three graspable cubes (red, green, blue) sit on a 20 cm × 20 cm table within Piper’s reachable workspace.

G Bedrock model identifiers and pricing

All LLM-agent experiments run against AWS Bedrock. Anthropic models use the native tool-use API; the other four vendors use the unified Bedrock Converse API through a thin adapter, so the agent loop is identical across vendors. Table 9 lists exact identifiers and per-token pricing used for all cost figures.

Across the leaderboard runs of §5.3 (seven models, using and synthesizing conditions), the agents consumed 25.4M input and 882k output tokens in total; per-configuration token counts are stored in each result JSON. The synthesis pipeline (§4) uses Sonnet 4.5 as its default agent. The cross-model capability study (§5.3) sweeps all seven of the other models above on the full 13-task SO-101 suite; smaller open-weight models (Ministral 8B, Qwen3 32B) and a strong open model (DeepSeek V3.2) are included to test whether synthesis is restricted to the strongest closed models. Gemma-3 and Llama-3/4 were excluded after a tool-use smoke test: Gemma-3-27B does not emit Converse tool calls, and the Llama endpoints required an inference profile not available in our account.

Figure 5 combines this pricing with the §5.3 leaderboard into a single cost–quality plane: each point is one model × method configuration over the full 13-task suite at $N=5$.

H Task execution filmstrips

Each row below is a single trial of the agent-synthesized driver executing a real task in MuJoCo, captured at five time points. The SO-101 Reach and Piper Pick strips are deterministic rollouts of the synthesized driver’s own `move_cartesian`

Label	Vendor	Bedrock identifier	Pricing (per 1M tok, in/out)
Haiku 4.5	Anthropic	us.anthropic.claude-haiku-4-5-20251001-v1:0	\$1.00 / \$5.00
Sonnet 4.5	Anthropic	us.anthropic.claude-sonnet-4-5-20250929-v1:0	\$3.00 / \$15.00
Sonnet 4.6	Anthropic	us.anthropic.claude-sonnet-4-6	\$3.00 / \$15.00
Opus 4.8	Anthropic	us.anthropic.claude-opus-4-8	\$5.00 / \$25.00
Nova Pro	Amazon	amazon.nova-pro-v1:0	\$0.80 / \$3.20
DeepSeek V3.2	DeepSeek	deepseek.v3.2	\$0.27 / \$1.10
Minstral 8B	Mistral	mistral.minstral-3-8b-instruct	\$0.10 / \$0.10
Qwen3 32B	Alibaba	qwen.qwen3-32b-v1:0	\$0.15 / \$0.60

Table 9: The eight Bedrock models (five vendors) and their per-token list pricing as of May 2026, used for all cost figures in the paper.

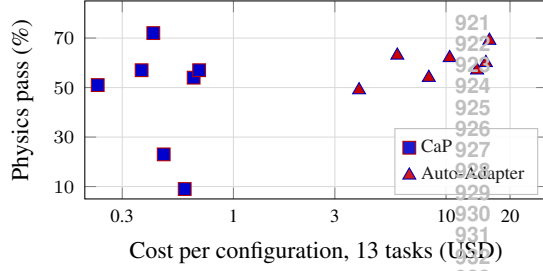


Figure 5: Cost–quality on the full 13-task SO-101 suite, seven models $\times$ two methods (§5.3). Triangles (ours) occupy a higher-success, higher-cost regime; squares (CaP) are cheap but high-variance (the high CaP outlier is Qwen3 32B). Cost is per 13-task configuration at $N=5$.

and gripper_close primitives. The SO-101 draw_letter_L strip re-executes the graded down-4 cm/forward-4 cm L trajectory of the hard-suite task (§5.3) through the synthesized driver’s own IK. The Go2, Skydio X2, H1, and ANYmal-C strips execute the respective drivers’ primitives live in MuJoCo (sit/stand_up; takeoff/move_to/land; squat; walk_forward), covering all four morphology classes the pipeline targets (Appendix L).

I Per-phase pipeline gallery

This appendix walks through one concrete pass of the §2 pipeline on SO-101, showing the actual artifact produced at each phase. All artifacts shown are reproduced verbatim from the SO-101 run logged in artifacts/auto_adapter_from_scratch_so101_artifacts/ (referenced throughout the body of the paper).

I.1 Phase 1: STUDY (parse MJCF, emit Spec)

The agent reads mjcf.xml, identifies actuators and joints, and writes a structured Spec object summarizing the robot’s degrees of freedom and limits. Excerpt:

```
{
```

```
"robot_id": "so101_from_scratch",
"estimated_class": "arm",
"dof": 5,
"arm_joint_names": ["shoulder_pan", "shoulder_lift",
                    "elbow_flex", "wrist_flex", "wrist_roll"],
"joint_limits": {
  "shoulder_pan": [-1.92, 1.92],
  "shoulder_lift": [-1.75, 1.75],
  "elbow_flex": [-1.69, 1.69],
  "wrist_flex": [-1.66, 1.66],
  "wrist_roll": [-2.74, 2.84]
},
"ee_site": "ee_site",
"ee_body": "gripper_link",
"gripper_actuator_names": ["act_jaw_visual"],
"actuator_type_hint": "position"
}
```

I.2 Phase 2: GENERATE (Spec $\rightarrow$ driver code)

The agent emits driver_from_scratch.py: 546 lines of self-contained Python that exposes home, move_cartesian, gripper_open, gripper_close, etc. The inverse-kinematics inner loop, written by the agent without any skeleton library, is reproduced below:

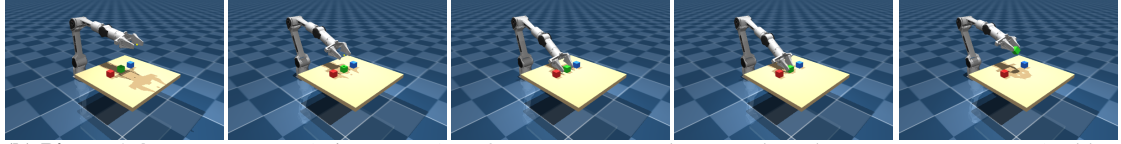
```
for iteration in range(max_iter):
    self.set_joint_positions(q)
    mujoco.mj_forward(self.model, self.data)
    current_xyz = self.data.site_xpos[self.site_id].copy()
    error = target_xyz - current_xyz
    if np.linalg.norm(error) < tol:
        return q, True, np.linalg.norm(error)

    J = self.compute_jacobian()
    # Adaptive damping: scales with error magnitude
    lambda_damping = self.ik_lambda_base * (1.0 + np.linalg.
        norm(error))
    JtJ = J.T @ J + lambda_damping**2 * np.eye(self.n_joints)
    dq = np.linalg.solve(JtJ, J.T @ error)
    # Step-size control prevents overshoot in poorly-
    # conditioned regions
    step = min(1.0, 0.5 / (np.linalg.norm(dq) + 1e-6))
    q += step * dq
    q = self._clamp_to_limits(q)
```

Damped Least Squares with adaptive λ and step-size clamping is a textbook IK formulation, here written from scratch by the agent against the per-robot Spec produced in Phase 1. The choice is deliberate rather than a limitation: DLS stays well-behaved near singularities and needs nothing beyond numpy, matching the self-contained delivery constraint (§2.3).



(a) **SO-101 reach_x+5**. Home $\rightarrow$ 33% $\rightarrow$ 66% $\rightarrow$ at target $\rightarrow$ return-to-home. $N=10$ trials, 10/10 pass (§5.1, Table 2).



(b) **Piper pick_green**. Home (gripper open) $\rightarrow$ 8 cm pre-grasp $\rightarrow$ lower to the cube $\rightarrow$ gripper_close (weld to cube) $\rightarrow$ lift 12 cm. The agent-written gripper_close does the distance check and weld activation; the 13-line post-synthesis patch adds get_object_position and the weld relpose update (§5.4).



(c) **SO-101 draw_letter_L** (hard suite; graded trajectory re-executed by the synthesized driver). A two-leg L within the SO-101 workspace: 4 cm down, then 4 cm forward (+X), then return home. Graded by ee_waypoint_trace, which verifies the end-effector actually visits both corners (in order) and returns home, not merely the return. $N=5$ per model, §5.3.



(d) **Go2 sit $\rightarrow$ stand_up attempt** (driver primitives executed live). The visible instability is the point: the driver passes the 8/8 onboarding validation (Table 5), yet the independent smoke suite scores stand_up 0/2 (Appendix C): the validator-leniency gap disclosed in Appendix B, and the reason every §5 number is graded by independent physics, never by the onboarding harness.

Figure 6: Task-execution filmstrips (manipulation and Go2). Five frames per row, captured at evenly-spaced time points during a single rollout of the agent-synthesized driver. Underlying simulator is MuJoCo (§3). All frames are produced from the same artifacts that drive the quantitative results in the body of the paper.

I.3 Phase 3: VALIDATE (run the harness)

The driver is imported and exercised against a fixed, robot-class-aware test suite. SO-101’s seven tests are reproduced verbatim from validate_report.json in Table 10:

Test	Status	Metric
no_skeleton_import	OK	—
robot_class_present	OK	—
build_from_mjcf	OK	—
home	OK	—
ik_roundtrip	OK	9.67 mm err
gripper_cycle	OK	—
describe	OK	—

Table 10: SO-101 validate-phase output (7/7); exact contents of artifacts/.../validate_report.json.

If any test fails, the orchestrator re-enters Phase 2 with the failure surfaced as a structured error in the prompt (Appendix F reports the controlled rerun showing the loop converges at outer iteration 1 on SO-101).

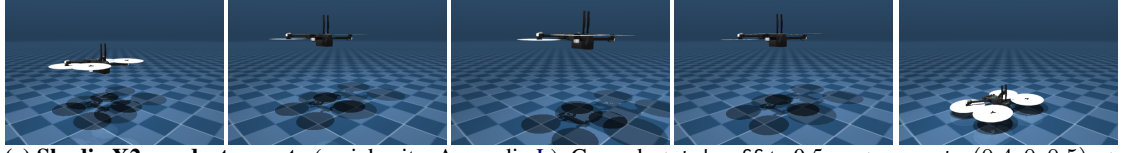
I.4 Phase 4: EXPORT (package the artifact)

A directory snapshot is captured for downstream consumption. The SO-101 artifact tree:

```
artifacts/auto_adapter_from_scratch_so101_artifacts/
|-- driver_from_scratch.py # 546 lines, agent-written
|-- mjcf.xml # symlink to assets/mjcf/so101_*.xml
|-- study.json # Phase 1 output (Spec)
|-- validate_report.json # Phase 3 output (test results)
-- IMPLEMENTATION_SUMMARY.md # agent-written narrative
```

I.5 Phase 5: DEMO (rendered execution)

The exported driver is exercised by a small demo script (home() then a reach target then home()) and the rollout is rendered frame by frame. Figure 8 shows the rollout at the moment the end-effector reaches its target. The full filmstrip for this same task is row (a) of Figure 6 in Appendix H; Appendix E gives a side-by-side Auto-Adapter vs. Code-as-Policies trace for one task.



(e) **Skydio X2 quadrotor go-to** (aerial suite, Appendix L). Ground $\rightarrow$ takeoff to 0.5 m $\rightarrow$ move_to (0.4, 0, 0.5) $\rightarrow$ return $\rightarrow$ land. The cascaded position $\rightarrow$ attitude $\rightarrow$ thrust controller is synthesized zero-shot from the MJCF; both agent styles score 8/8 on the hardened aerial suite (Table 17).



(f) **H1 humanoid squat-recover** (Appendix L). The 19-DoF biped holds a PD-balanced stand, bends at the knees and hips, and recovers without falling over; the motion is deliberately conservative (torso dips from 0.98 m and returns to 0.97 m), which is what the validated squat primitive does; dynamic walking remains the pipeline’s hardest open case (Appendix L).



(g) **ANYmal-C walk_forward** (locomotion v2 suite, §5.4). Stand, then four 2 s walk_forward calls at speed 0.2, the same call pattern as the graded eval trials; the base advances 0.24 m while holding ≈ 0.5 m body height. Executed live on the shared driver behind the §5.4 locomotion comparison, where ANYmal-C favors Auto-Adapter (68% vs. 28%).

Figure 7: Morphology-breadth filmstrips (aerial, humanoid, quadruped walking), rendered as in Figure 6.

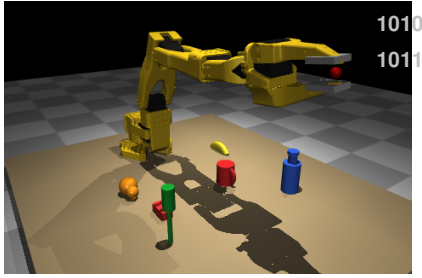


Figure 8: SO-101 demo frame at reach_x+5 target position (end-effector at home+5 cm in X, in MuJoCo). The synthesized driver’s move_cartesian produced the trajectory. Compare against Figure 6(a) for the full rollout.

J Grader taxonomy

Suite	Loc.	Robot	Tasks
Cross-skill	5.1	SO-101	4 reach, 10 pick
Multi-task	5.2	SO-101	6 dirs + 4 held-out
Hard suite	5.3	SO-101	12 phys + 1 cov
Simple suite	5.4	Piper, UR5e	5 short tasks
Piper Pick	5.4	Piper	3 reach + 3 pick
OpenVLA reach	K.1	SO-101	5 reach (RGB)
Locomotion v2	5.4	Go2, ANYmal-C	10 loco tasks

Table 11: Downstream task suites (§5). Reach suites run 10 trials per offset; the hard suite is 12 physics-graded plus 1 coverage task.

Grader taxonomy. AutoAdapter-Bench implements 42 grader types in eval.py; 31 are exercised by the released task suites. Twenty-eight of those

31 read *post-step simulator state*: end-effector pose (ee_at_midpoint, ee_waypoint_trace), object pose and contact (object_above_object, object_lifted_by, objects_swapped, object_pose_match_relative), body height (body_height_ratio), and base pose/yaw for locomotion (forward_progress, turn_to_heading, drive_and_turn). The remaining three are deliberately *behavioral*, and we label them as such rather than fold them into the physics count: failure-detection (llm_reports_failure, used by unreachable_detect, where the correct behavior is to *report* an IKUnreachableError rather than move), tool-liveness (tool_executed_without_crash, used by gripper_cycle), and end-effector coverage (ee_visited_n_positions, used by visit_all_objects). The first two sit in the simple suite (§5.4); only the third, visit_all_objects, sits in the 13-task hard suite. In the released code ee_visited_n_positions now verifies that the end-effector physically came within tolerance of each object’s true world position (falling back to commanded-target coverage only when the per-call trajectory is unavailable).

Grasp grading. For grasp tasks, object_lifted_by requires the target to be

both lifted *and* still held within 12 cm of the end-effector at the final step, rejecting an object flung by a faulty weld constraint that would pass a height-only check. The self-contained validation suite includes a matching grasp_lift test, so a driver that cannot truly grasp fails synthesis rather than inflating its Self score.

K 12-task robustness and statistical tests

12-task physics-only robustness. The single behavioral task in the hard suite is visit_all_objects. Table 12 reports the SO-101 portability comparison on the 12 physics-graded tasks alone (dropping that one task), using the same trials. The model ranking is unchanged: Auto-Adapter beats zero-shot CaP on six of seven models and beats few-shot CaP on four (tying Haiku), exactly as on the full 13-task suite (Table 3). The headline therefore does not depend on the behavioral task or on how it is graded.

Model	A-A	CaP	CaP _{fs}	Self
Sonnet 4.6	67	53	50	–
Opus 4.8	60	47	45	42
Haiku 4.5	58	50	58	42
Nova Pro	57	53	42	–
DeepSeek V3.2	53	10	57	22
Ministral 8B	50	25	38	–
Qwen3 32B	45	70	53	–

Table 12: SO-101 portability on the **12 physics-graded** hard-suite tasks (visit_all_objects excluded), %, $N=5$ trial-level. Bold = best of A-A/CaP/CaP_{fs}. Ranking matches the 13-task Table 3: A-A > CaP on 6/7; A-A > CaP_{fs} on 4/7 (Haiku tie).

Statistical significance. In brief: the Auto-Adapter advantage over zero-shot CaP is significant at the task level, but individual per-model cells are not separately resolvable at $N=5$. We test Auto-Adapter vs. zero-shot CaP with the (model, task) cell as the paired unit (seven models $\times$ 13 tasks, $n=91$ cells; $N=5$ trials each), computed by autoadapter_bench/stats.py. Auto-Adapter leads by a mean +13.0 points per cell, and a Wilcoxon signed-rank test over the paired cells is significant ($W=138$, $p<0.01$; pooled trial rate 0.59 vs. 0.46). The coarser model-level comparison over the seven aggregate rates is only suggestive (+13 pp mean; sign test $p=0.13$, Wilcoxon $p=0.15$; $n=7$ is underpowered), so we rest the claim on the task-level test rather than on the 6/7 win count. Against few-shot CaP the aggregate effect is *not* significant ($p=0.13$), so we report that

comparison descriptively only. Per cell, only the two largest gaps clear non-overlapping 95% Wilson intervals (DeepSeek A-A 0.57 [0.45, 0.68] vs. CaP 0.09 [0.04, 0.19]; Ministral A-A 0.54 [0.42, 0.65] vs. CaP 0.23 [0.15, 0.35]); for the four closely matched Anthropic/Nova cells the per-model intervals overlap. The effect is therefore robust in aggregate but, at $N=5$, individual cell differences below ~ 15 points are not separately resolvable, so we do not claim per-model significance.

Quadruped agent-model ablation. The §5.4 quadruped numbers use a Sonnet 4.5 agent on a shared Sonnet 4.5 driver. Re-running the same 10-task suite with a Sonnet 4.6 agent gives the same qualitative split (ANYmal-C favors Auto-Adapter, Go2 reverses), consistent with an embodiment-driven rather than agent-version-driven effect (Table 13).

Robot	Agent	A-A	CaP
ANYmal-C	Sonnet 4.5	68	28
ANYmal-C	Sonnet 4.6	62	50
Go2	Sonnet 4.5	48	54
Go2	Sonnet 4.6	44	54

Table 13: Quadruped locomotion (10-task v2 suite, physics %, $N=5$): Sonnet 4.5 (main, §5.4) vs. Sonnet 4.6 (ablation). The ANYmal-favors-A-A / Go2-reverses split holds under both agent versions.

K.1 Pretrained-VLA probe: OpenVLA-7B zero-shot on SO-101

As an auxiliary check we ran a pretrained vision-language-action model, OpenVLA-7B, zero-shot on the five SO-101 reach variants (50 episodes). This is a cross-embodiment *probe*, not a like-for-like baseline: OpenVLA reads RGB rather than the privileged state our agent and the RL/CaP baselines use, SO-101 is not in its Open-X training mixture, and its camera frame differs from training; we therefore keep it out of the main comparison. It scores **0/50** (mean EE error 23 cm, deltas in inconsistent directions), consistent with the per-embodiment finetuning OpenVLA requires (Kim et al., 2024, §3). Because a zero invites the objection that the action adapter, not the model, failed, we validate the harness and sweep the two configuration knobs a reviewer would question. *Harness validation:* an oracle that feeds the ground-truth direction (target – current EE) through the identical move_cartesian path reaches all five reach targets (5/5, mean final error 1.3 cm) at the per-step control budget used for OpenVLA (duration=0.5 s); the

same path with a shorter duration (0.1 s) reaches 0/5. A capable controller therefore passes on this harness, so a model score of 0 is a model result. **Robustness:** Table 14 sweeps six action scales (0.25–10×) and eight unnorm de-normalization keys OpenVLA-7B ships (bridge_orig plus seven others, the manipulation datasets closest to SO-101). Every cell is $\leq 6.7\%$ (one chance episode of 15), so the failure is not an artifact of one de-normalization or scale choice.

unnorm key	scale	success	err (cm)
bridge_orig	0.25	0.0%	10.3
bridge_orig	0.50	0.0%	13.1
bridge_orig	1.00	6.7%	15.3
bridge_orig	2.00	0.0%	23.8
bridge_orig	5.00	0.0%	33.7
bridge_orig	10.0	0.0%	47.0
fractal20220817_data	2.00	0.0%	58.4
berkeley_autolab_ur5	2.00	0.0%	18.7
taco_play	2.00	0.0%	50.2
jaco_play	2.00	0.0%	54.2
viola	2.00	0.0%	58.2
toto	2.00	0.0%	16.7
austin_buds	2.00	0.0%	57.6

Table 14: OpenVLA-7B config-robustness sweep on SO-101 (3 episodes $\times$ 5 reach variants per cell; success = mean per-variant rate). Top: action-scale sweep at the canonical key. Bottom: unnorm-key sweep at the canonical scale. Harness oracle-validated above.

K.2 Franka reach per-variant breakdown

Table 15 gives the per-variant Franka reach results behind §6 (20/70 overall).

Variant	Pass	Mean err (cm)
reach_x+5	10/10	1.76
reach_z+5	10/10	0.13
reach_x-5, y $\pm$ 5, diag	0/40	2.86
reach_z-5	0/10	11.07
Total	20/70 (28.6%)	3.49

Table 15: Franka reach: 7 variants $\times$ 10 trials; x-5/y $\pm$ 5/diag aggregated, z-5 (workspace boundary) separate.

Oracle ablation: solver or synthesis? To separate the agent from the control template, a deterministic scripted oracle issues the *exact* Cartesian target for the same seven variants under the same methodology and 2 cm tolerance — no LLM anywhere. The oracle reproduces the published failure signature exactly (2/7; 2–4 cm on the same four directions; 11.1 cm on z-5), so the failures are deterministic template behavior, not agent errors or synthesis noise (determinism check: the stock and

5 $\times$ -iteration arms produce bit-identical per-variant errors). Neither a 5 $\times$ IK iteration budget nor a nullspace-regularized DLS variant repairs it (both 2/7), so a stronger solver configuration alone is not the fix (Table 16). A per-variant decomposition shows two mechanisms: z-5 is a true IK failure at the workspace/joint-limit boundary, while three of the four 2–4 cm cases have sub-millimeter IK residuals (x-5 is mixed, 1.1 cm) and lose accuracy in the execution layer (in these reaches, which stay near the arm’s default configuration, the wrist joint settles $\approx 11^\circ$ from its commanded value). Consistent with an execution-layer component, gravity-compensated execution of the same stock IK solutions recovers one variant (3/7). This does not change the reported failure pattern or the ceiling claim; it narrows the cause from solver choice alone to the broader prescribed arm-control template.

Arm (deterministic, no LLM)	Pass	Note
Stock DLS driver	2/7	same signature as Table 15
Stock, 5 $\times$ iterations	2/7	bit-identical errors
Nullspace-regularized DLS	2/7	fixes x-5, loses x+5
Stock + gravity-comp. execution	3/7	recovers diag

Table 16: Franka oracle ablation: a scripted oracle replays the exact targets. The published failures persist with no LLM in the loop and are not repaired by solver changes alone.

L Morphology breadth: aerial and humanoid

The §4 scoreboard covers serial arms and quadrupeds. To test whether the *same* pipeline (unchanged except for a class-specific generation prompt and physics validator) generalizes to qualitatively different dynamics, we onboard two further morphologies from MuJoCo Menagerie: a free-flying quadrotor (Skydio X2) and a bipedal humanoid (Unitree H1). Both are floating-base and open-loop unstable, so the synthesized driver must contain a stabilizing feedback controller, not just kinematics. We keep these two outside AutoAdapter-Bench proper (the eight robots of §3) and report them as breadth tests instead: on each we run driver synthesis and the same-API CaP comparison, but not the learned baselines (PPO, Diffusion Policy, OpenVLA) and seven-model leaderboard that anchor the benchmark, so folding them into the benchmark count would overstate how much head-to-head evidence backs them.

Aerial (Skydio X2). The agent synthesizes a driver exposing takeoff, move_to, hover, land, and

get_base_pose; internally it implements a cascaded position $\rightarrow$ attitude $\rightarrow$ rotor-thrust controller for the underactuated 4-rotor base. It passes the aerial validation suite (7/7: build, home, takeoff to commanded altitude while upright, and move_to to a world target without flipping) on a Sonnet 4.5 agent. On an 8-task aerial suite (4 simple + 4 hard; takeoff-to-altitude, hover, point-to-point, climb, round-trip, diagonal, square patrol, tight hover; physics-graded on post-step base pose with an upright constraint, and hardened so that a do-nothing agent scores 0/8: round-trips require real movement, hovers require the commanded altitude, the patrol’s waypoint tolerance is below the square’s center-to-corner distance), Auto-Adapter and same-API CaP *both* reach 100% (Table 17, $N=5$; filmstrips in Figure 7). Unlike contact-rich manipulation (§5.3), where the multi-turn loop confers an advantage, point-to-point flight through the synthesized high-level API (takeoff, move_to) is solvable by a single-shot program, so A-A and CaP tie. The aerial contribution is therefore the *validated flight controller itself* (a cascaded thrust stabilizer synthesized zero-shot from the MJCF), not an agent-loop win.

Aerial task	A-A	CaP
takeoff to 0.5 m	✓	✓
hover at 0.5 m	✓	✓
go to (0.4, 0, 0.5)	✓	✓
climb to 1.0 m	✓	✓
out-and-return	✓	✓
diagonal go-to	✓	✓
square patrol (4 wpts)	✓	✓
tight hover	✓	✓
Physics % ($N=5$)	100	100

Table 17: Skydio X2 aerial suite on the agent-synthesized drone driver (Sonnet 4.5). ✓ = 5/5 trials pass. Both agent styles solve all eight on the hardened suite (a do-nothing controller scores 0/8): point-to-point flight through the synthesized high-level API is single-shot solvable, so Auto-Adapter and CaP tie; the contribution is the validated flight driver, not the agent loop.

Humanoid (Unitree H1). The 19-DoF H1 is a tall inverted pendulum: open-loop it falls. The agent synthesizes a joint-space PD balance controller that passes onboarding validation 7/7: it holds a *stable stand* (torso 0.98 m, upright 1.00 on the body- z axis for the full hold) and *squat-recover* without falling over (torso dips then returns to 0.97 m at upright 0.99). The result is reproducible under the

benchmark protocol (home-initialized, as in validation and replay): stand 4/4 and squat-recover 3/3 across re-runs. Synthesizing a validated stand-and-squat balance controller for a 19-DoF biped zero-shot from the MJCF is, to our knowledge, the hardest morphology the pipeline handles. We report the headline H1 driver at the onboarding level (the orchestrator steps physics directly for stand and squat), since a pure *stand* replay can be passed by doing nothing (a homed biped stays up if physics is not advanced); the cross-model sweep below instead grades H1 on *active* stand-and-squat tasks, where a squat must lower the body and return, so a do-nothing controller fails. We also *attempted* dynamic walking: the generator was prompted for a closed-loop micro-step gait, validator-certified for forward progress in the starting direction, staying upright, and bounded sideways/turning drift (so falling forward, standing still, and spinning cannot pass). It did not yield a stable validator-certified walk across attempts: adding the walking objective even broke the otherwise-reliable stand. Dynamic bipedal locomotion is thus the pipeline’s hardest open case: static balance and squat are synthesizable zero-shot, dynamic gait is not (yet).

Cross-model self-synthesis across morphologies.

The §5.3 “writes API” result (only the three strongest models synthesize a working SO-101 grasp driver) raises a question: is self-synthesis difficulty a property of the *model* or of the *morphology*? We ran the from-scratch pipeline for all seven LLMs on three non-gripper morphologies (Go2 quadruped, Skydio X2 quadrotor, Unitree H1 humanoid), two attempts each, scoring a cell as a pass only if the synthesized driver clears the class-specific validator and is not hard-coded (no skeleton import, no literal leg-name constants). Table 18 reports reliability; Table 19 the task success of the passing drivers. Two patterns hold. (i) *Reliability is morphology-ordered*: more models clear aerial flight than locomotion than the humanoid biped, and only the two strongest models in this sweep (Opus, Sonnet) synthesize across more than one morphology while the open/small models (Nova, Ministral, Qwen) clear none. (ii) *Task success of the drivers that do validate is also morphology-ordered*: flight 0.82–1.00 $\gtrsim$ locomotion 0.70–0.82 $>$ humanoid 0.50 $>$ SO-101 grasp 0.28–0.46 (§5.3). Self-synthesis is thus jointly gated by model capability *and* morphology: in task success, writing a correct low-level controller gets harder from smooth flight, through

stepping locomotion and slow biped balance, to contact-heavy grasping; in synthesis reliability the biped is the rarest to validate (Table 18).

	Op	So	Ha	No	DS	Mi	Qw
Skydio X2 (fly)	2/2	2/2	1/2	0/2	1/2	0/2	0/2
Go2 (walk)	1/2	1/2	0/2	0/2	0/2	0/2	0/2
H1 (biped)	1/2	0/2	0/2	0/2	0/2	0/2	0/2

Table 18: Cross-model self-synthesis **reliability** (validated, non-hardcoded driver / 2 attempts). Op=Opus 4.8, So=Sonnet 4.6, Ha=Haiku 4.5, No=Nova Pro, DS=DeepSeek V3.2, Mi=Ministral 8B, Qw=Qwen3 32B.

	Opus	Sonnet	Haiku	DeepSeek
Skydio X2 (fly)	1.00	0.95	1.00	0.82
Go2 (walk)	0.70	0.82	–	–
H1 (biped)	0.50	–	–	–

Table 19: **Task success** (physics, $N=5$, simple+hard) of the self-synthesized drivers that passed validation; – = no validated driver (synthesis failed). Models that validated on no morphology (Nova, Ministral, Qwen) are omitted.

M Lessons from a physical-deployment attempt

To scope what separates the simulation results from a deployable system, we ran one bring-up session on a physical SO-ARM101 (Feetech STS3215 servos, wrist-mounted RealSense D405, Claude Opus 4.8 as the perception VLM). The same Generate phase synthesized a real-robot driver from the SO-101 MJCF/URDF and a one-line spec. The arm reached a kinesthetically taught grasp pose within 2 mm after a one-shot servo-bias correction, and the VLM localized the target cube in the eye-in-hand view; end-to-end autonomous grasping of a 4 cm cube still failed. The failures break down into four independent gaps.

(1) Servo reproducibility. Commanded vs. actual joint positions differ by 1–4° per joint under gravity load, propagating to 5–8 mm of end-effector error. A feed-forward bias vector recovers 2 mm accuracy but is pose-dependent: one compensation vector does not generalize across the workspace.

(2) Hand-eye calibration. The wrist-mounted camera-to-base transform must be calibrated per arm. Our perturbation-based grid searches were ill-conditioned (13–17 cm predicted-depth error); the working industry practice is a human-placed

calibration target with a stored affine solve, which is automatic only *after* the human places markers.

(3) Depth sensing at close range. With the end-effector at 24 cm, scene depths of 6–15 cm sit at the lower edge of the D405’s working range; the gripper fingers themselves read as invalid depth, and glare on metal surfaces corrupts detection boxes.

(4) Grasp verification. The physical arm has no force, tactile, or slip sensing; grasp success must be inferred from a single gripper-encoder integer. That signal is too coarse to drive the outer verify-and-retry loop that every simulation result in this paper relies on.

None of the four gaps is specific to LLM-written code; a hand-written driver would face the same per-arm bring-up. What simulation removes is precisely the feedback channel (§2.2) that Auto-Adapter’s validation depends on. The practical boundary is therefore: simulator-grounded synthesis automates the *algorithmic* layer of onboarding (kinematics, control, primitive API), while calibration, bias compensation, and contact sensing remain a per-robot engineering layer that today takes a human a session of bring-up time.

Safety requirements for any physical extension.

In this paper the write–execute–revise loop never touches hardware: all generation, execution, and revision happen in MuJoCo, and the exported driver is a static, auditable artifact. The bring-up session above used conservative manual operation with z-bounds and joint-limit checks in the bridge. Before generated code executes on hardware we consider the following minimally necessary: (i) non-overridable driver-level clamps on joint limits, velocity, and torque; (ii) a workspace bounding volume enforced outside the generated code; (iii) sim-first re-validation of any revised driver before hardware execution; (iv) collision and force monitoring with automatic stop; (v) a hardware e-stop; and (vi) a human approval gate with staged, slowed rollout for the first hardware run of any newly generated driver.

Future work: scaling physical validation. Our concrete next step is perception-grounded reach and place on this physical SO-ARM101 with the wrist-mounted RGB-D camera, executed under the safety requirements above (sim-first re-validation, driver-level clamps, human approval gate), taking the four gaps documented in this appendix as the work items; a controlled study of an engineer using LLM coding assistants for the same onboarding

task is a second open comparison.

1346

N Complete system prompts

This appendix reproduces, verbatim from the released code, the four system prompts behind every result in the paper: the two synthesis phases (Study, Generate), the Demo-phase task planner, and the Code-as-Policies baseline. Validate is a deterministic harness, not an LLM call, so it has no prompt. Unicode glyphs are transliterated to ASCII for typesetting; no other edits. One exception: the Piper pickbench run (§5.4) predates the explicit `get_object_position` requirement shown in the Generate prompt, which is why that driver needed the 13-line post-synthesis patch. Per-task user messages are the task prompts listed in the released `autoadapter_bench/spec/tasks_*.yaml`.

N.1 Phase 1 (Study) system prompt

Used verbatim for every robot; the agent fills the schema by inspecting the MJCF through the mujoco API.

```
You are Phase 1 STUDY. Read the robot's MJCF and produce study.json with the
kinematic + actuator + grasp information the next phase needs to write a
driver from scratch.

IMPORTANT: keep each execute_python call short (<=150 lines). The session
keeps state across calls.

Procedure:
1. read_file the MJCF.
2. execute_python: parse with mujoco.MjModel.from_xml_path; for each
   ARM joint (hinge/slide that the IK should move) record name, axis,
   ctrlrange, qpos limits, parent body. Identify the end-effector
   site OR body. Identify gripper actuator(s) if any. Identify any
   weld equality constraints between gripper body and free-joint bodies.
3. execute_python: classify the robot (arm | quadruped | wheeled | aerial | humanoid | ...).
4. write_file study.json with this exact schema:
{
  "robot_id": "<id>",
  "estimated_class": "arm|quadruped|...",
  "dof": <int>,
  "arm_joint_names": [...],
  "arm_actuator_names": [...],
  "joint_limits": {name: [lo, hi]},
  "ee_site": "<name or null>",
  "ee_body": "<name or null>",
  "grripper_actuator_names": [...] or null,
  "grripper_ctrlrange": [lo, hi] or null,
  "weld_graspable_bodies": [...] or null,
  "actuator_type_hint": "position|velocity|motor|tendon",
  "notes": "<one-line summary>"
}
5. Reply with one sentence confirming the file.
```

N.2 Phase 2 (Generate) system prompt, self-contained mode

One prompt covers all five morphology classes; the class-specific block is selected by the robot class the agent itself recorded in `study.json`. This is the complete text; no robot-specific variant exists.

```
You are Phase 2 GEN_ALGO. You will write driver_from_scratch.py -- a complete,
single-file Python driver for this robot -- WITHOUT importing anything from
auto_adapter.skeletons. The framework provides building blocks; you choose
the algorithms.

Hard rules:
- Use `mujoco`, `numpy`, and Python stdlib only. NO `from auto_adapter.skeletons
  import *`. Don't copy framework classes wholesale.
- The driver must expose a class `Robot` with classmethod
  `build_from_mjcf(mjcf_path: str) -> Robot` plus `home() -> bool`,
  `get_joint_positions()`, `step(n: int = 1)`, `render()`, `describe()`.
- Beyond those, the API should fit the robot's CLASS (from study.json):

  ARM (serial chain with end-effector):
  - get_ee_pose() -> (xyz, R3x3)
  - move_cartesian(target_xyz, duration=2.0) -> bool
  - gripper_open() / gripper_close() -> bool (if gripper exists)
  - is_holding() -> bool
  - get_object_position(name: str) -> xyz (REQUIRED for manipulation:
    world position of a named scene object. The task planner will name
    objects, e.g. "banana", "mug", "cube_red". Resolve the name to a
    body -- fall back to geom, then site -- via mujoco.mj_name2id and
    return its world xyz from mjData (data.xpos / geom_xpos / site_xpos).
    Raise KeyError if the name is unknown.)
  - get_object_names() -> list[str] (names of the manipulable scene
    objects, so a planner can discover what's in the world.)
  Pick an IK method that fits DOF + redundancy:
```

```

* 5-DOF non-redundant -> DLS or Newton-Raphson
* 6-7 DOF redundant -> DLS with nullspace, or pseudoinverse + secondary
* Spherical wrist -> analytical IK if you can derive it
Robust IK: clamp to joint limits, raise on unreachable, scale damping
with residual.

QUADRUPED (4 legs, locomotion, torque actuators):
DO NOT WRITE IK -- there's no end-effector to reach. Write:
- stand_up(duration=2.0) -> bool (PD to a STABLE standing pose;
  body height must end > 0.15 m and stay there)
- sit(duration=1.5) -> bool (PD to a folded pose that ACTUALLY
  LOWERS the body: final height must drop below 0.8x the standing
  height -- fold hips+knees, do not just hold the stand pose)
- walk_forward(secs=2.0, speed=0.2) -> bool (gait that produces REAL
  forward base displacement: >3 cm over ~1.5 s while staying upright,
  with small sideways drift -- tune the gait, a degenerate in-place
  shuffle will fail validation)
- get_body_height() / get_base_pose()
Use joint-space PD ( $\tau = k_p(q_{des} - q) + k_d(\dot{q}_{des} - \dot{q})$ ). For walking,
pick a simple periodic gait -- diagonal pairs swing anti-phase -- and
verify forward progress in your local_exec self-test before finishing.
These three behaviors are VALIDATED on actual physics (height drop for
sit, forward displacement for walk), not just "did not crash".

WHEELED / MOBILE BASE (differential drive, wheel-velocity actuators):
DO NOT WRITE ARM IK. The base moves via wheel-velocity actuators
(e.g. left_wheel_vel / right_wheel_vel). Write:
- drive_forward(distance_m=0.3, speed=1.0) -> bool (BOTH wheels same
  velocity; produce REAL forward base displacement, low sideways drift;
  close the loop on get_base_pose to stop near distance_m)
- turn(angle_rad, speed=1.0) -> bool (DIFFERENTIAL wheel velocities;
  change base YAW by the requested angle, verify via get_base_pose)
- get_base_pose() -> (xyz, R3x3) (world base position + orientation)
- get_base_yaw() -> float
drive_forward and turn are VALIDATED on physics (real forward
displacement; real yaw change), not "did not crash". Tune wheel speed
and step count and verify in your local_exec self-test before finishing.

AERIAL / MULTIROTOR (free-flying base, N thrust actuators):
DO NOT WRITE ARM IK and DO NOT WRITE A WALKING GAIT. A quadrotor is
UNDERACTUATED and open-loop UNSTABLE -- it WILL flip and crash unless a
feedback controller runs every sim step. Write:
- takeoff(height=0.5) -> bool (spin rotors up and climb to ~height m
  altitude, then HOLD a stable hover; final z must be near height and
  the body must stay upright -- a flip/crash fails validation)
- move_to(x, y, z, tol=0.1) -> bool (fly to a world target with a
  closed loop on get_base_pose; arrive within tol and stay upright)
- hover(secs=2.0) -> bool (station-keep at the current pose)
- get_base_pose() -> (xyz, R3x3) (world base position + orientation;
  use this exact name -- downstream tooling reads base pose through it)
- land() -> bool (descend to the ground and idle the rotors)
Use a CASCADED controller stepped each mj_step: an outer position loop
(PD on x,y,z error -> desired total thrust + desired roll/pitch angles)
and an inner attitude loop (PD on roll/pitch/yaw -> per-rotor thrust
offsets), then mix to the n thrust actuators. Per-rotor hover thrust is
about total_mass * 9.81 / n_rotors (clamp to actuator ctrlrange). Read
body orientation from the free-joint quaternion (data.qpos[3:7]) and
body rates from data.qvel[3:6]. VERIFY in your local_exec self-test that
the drone holds altitude for >=1 s and reaches a moved target WITHOUT
flipping -- a controller that drifts or tumbles fails validation.

HUMANOID / BIPED (free-floating torso, many joint actuators, legs+arms):
DO NOT WRITE ARM IK. A humanoid is a tall inverted pendulum
-- open-loop it falls. Bipedal balance is the hard part; prioritise
STATIC balance over dynamic walking. Write:
- stand_balance(secs=3.0) -> bool (hold a STABLE standing pose for
  `secs`: PD-track a sensible nominal joint configuration -- slightly
  bent knees/hips, torso vertical -- and keep the torso upright and at
  roughly its standing height the whole time; falling fails validation)
- squat(depth=0.15, secs=3.0) -> bool (smoothly lower the torso by
  ~depth m by bending hips+knees, then return to standing; stay
  balanced and recover the standing height -- do not topple)
- get_base_pose() -> (xyz, R3x3) (torso/pelvis world position +
  orientation; use this exact name)
- get_torso_height() -> float (torso world Z)
- walk_forward(distance_m=0.10, secs=2.0) -> bool (ATTEMPT a small
  forward walk; ONLY try this once stand_balance is robust. Dynamic
  bipedal walking is HARD -- a 5-10 cm shuffle is a success, do not
  aim far.)
Use joint-space PD ( $\tau = k_p(q_{des} - q) + k_d(\dot{q}_{des} - \dot{q})$ ) with per-joint
gains; read the torso orientation from the free-joint quaternion
(data.qpos[3:7], wxyz order). Define the nominal standing pose from the
MJCF keyframe if present, else a mild crouch. VERIFY in your local_exec
self-test that the torso stays up for the full duration and that squat
lowers then recovers WITHOUT falling.
WALK (only if you attempt it) -- make it a CLOSED-LOOP quasi-static
micro-step state machine, NOT open-loop sinusoids:

```

1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495

1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506

```

1507         * 4 phases: shift weight onto stance foot -> lift+place swing foot a
1508             small step (2-4 cm, 1-2 cm clearance) -> settle into double support
1509             -> swap stance/swing. Step period ~0.5-0.8 s.
1510         * Keep the torso over the stance foot before unloading the swing foot;
1511             feet parallel, NO cross-over, NO large yaw change, slight crouch.
1512         * Update q_des ONLINE every control substep from the balance state
1513             (base pose/vel, torso height, foot poses, contact if available);
1514             smooth/interpolate targets; clamp to joint+actuator limits.
1515         * ABORT immediately (return without falling further) if torso height
1516             or uprightness drops below a guard. Forbidden: open-loop joint
1517             oscillations, a large first step, writing the base pose directly,
1518             judging progress by world-x (use the start-heading frame).
1519         It is FINE if walk_forward fails -- stand+squat already validate; an
1520         honest "walk not achieved" is acceptable. Do NOT fake forward progress.
1521
1522     OTHER (biped, ...):
1523         Whatever makes sense. Write the behaviors a useful task planner
1524         would call. Don't force-fit arm semantics on a non-arm robot.
1525
1526     - Code should be ROBUST: graceful failure, clamp to limits, helpful error
1527       messages.
1528
1529     You have these tools:
1530     - read_file (study.json, mjcf.xml, mujoco source under .venv if useful)
1531     - execute_python (sandboxed CI -- for prototyping math, parsing MJCF, etc.)
1532     - local_exec (LOCAL -- `python` resolves to the venv with mujoco; use this
1533       to actually test the driver you wrote)
1534     - write_file (workspace-relative)
1535
1536     Procedure:
1537     1. read_file study.json.
1538     2. execute_python or local_exec: prototype FK using mujoco -- set qpos,
1539       call mj_forward, read site_xpos / body xpos. Verify your understanding
1540       of the index resolution works.
1541     3. execute_python or local_exec: prototype your IK on the home pose +
1542       a small offset. Iterate until residual < 1cm. PRINT the algorithm
1543       details (damping schedule, iteration count, step clamp).
1544     4. write_file driver_from_scratch.py -- the FULL Robot class, well-commented,
1545       with the algorithms you just prototyped baked in.
1546     5. local_exec: `python -c "import driver_from_scratch; r =
1547       driver_from_scratch.Robot.build_from_mjcf('mjcf.xml'); r.home();
1548       print(r.describe())"` to prove it constructs + runs.
1549     6. local_exec: test ik_roundtrip -- move EE +5cm in X, read back, error < 2cm.
1550     7. Reply with one sentence summarizing: which IK method you chose,
1551       final residual on the round-trip test, and a brief description of
1552       gripper/grasp behavior.

```

1553 N.3 Demo-phase task-planner system prompt

1554 {cls_name}, {tool_names}, and {class_hint} are filled at runtime from the synthesized driver's
 1555 describe() output, so the planner prompt never encodes robot-specific structure.

1556 You are the OPERATOR of a robot. The user gives you a task in English and you
 1557 accomplish it by calling tools that act on a {cls_name}. The world is
 1558 deterministic and reset before each task.

1559 Available tools: {tool_names}.

1560 {class_hint}

1561 Procedure for any task:

- 1562 1. Use observation tools (get_ee_pose / get_object_position / etc.) to
- 1563 understand the current state -- don't guess positions.
- 1564 2. Plan a sequence of motions / gripper ops that accomplishes the task.
- 1565 3. Execute them, OBSERVING after each action that needs feedback
- 1566 (e.g. after gripper_close, check is_holding before lifting).
- 1567 4. When done, reply with one sentence summarizing what you accomplished
- 1568 and any relevant final measurement.

1569 Tips:

- 1570 - All coordinates are in WORLD frame, meters.
- 1571 - Plan in small, observable steps -- don't fire 10 tools without checking
- 1572 intermediate state.
- 1573 - If a motion fails (returns False or IK unreachable), don't repeat it;
- instead query the EE pose and try a smaller / different target.
- The video record is automatic; you don't need to render or save anything.

1574
1575
1576
1577
1578
1579

1580 N.4 Code-as-Policies baseline system prompt

1581 The baseline sees the same primitives as Auto-Adapter's planner but must emit one single-shot program;
 1582 the primitive list shown is the complete class-conditional menu. For arms the baseline-facing move_to is
 1583 a thin alias over the driver's move_cartesian (identical semantics; same frozen driver as Auto-Adapter),
 1584 so the two operate the same API under different surface names.

You are a robot programmer. The user gives you a task in English; you reply with ONE Python program that uses ONLY the following primitives to accomplish the task. The program runs in a sandbox where these primitives are already bound as module-level functions. Imports of `numpy` (as `np`), `math`, and `time` are pre-loaded.

OUTPUT FORMAT: a single fenced Python code block (``python ... ``), no prose before or after. No `print` calls -- return final state via the primitives' side effects on the robot.

AVAILABLE PRIMITIVES (these are the only functions you may call):

ARM (5-7 DOF serial manipulator):

```
home() -> bool
    Move arm to home configuration.
get_ee_pose() -> (np.ndarray xyz, np.ndarray R3x3)
    Current end-effector world-frame pose.
move_to(xyz, duration=2.0) -> bool
    Move end-effector to target xyz in world frame.
    Raises RuntimeError on unreachable target.
gripper_open() -> bool
    Open the gripper. Releases any held object.
gripper_close() -> bool
    Close the gripper. Returns True iff an object was grasped.
is_holding() -> bool
    Whether the gripper has an object.
get_object_position(name: str) -> np.ndarray
    World-frame xyz of a body. Common names depend on the scene
    (banana, mug, bottle, screwdriver, duck, lego for S0-101).
get_joint_positions() -> np.ndarray
    Current arm joint angles.
```

QUADRUPED (4-leg locomotion robot):

```
stand_up(duration=2.0) -> bool
sit(duration=1.5) -> bool
walk_forward(secs=2.0, speed=0.2) -> bool
get_body_height() -> float
get_base_pose() -> (np.ndarray xyz, np.ndarray R3x3)
get_joint_positions() -> np.ndarray
```

WHEELED (differential-drive mobile base):

```
drive_forward(distance_m=0.3, speed=1.0) -> bool
    Drive straight forward by distance_m meters.
turn(angle_rad, speed=1.0) -> bool
    Turn in place by angle_rad radians (+ = left/CCW).
get_base_pose() -> (np.ndarray xyz, np.ndarray R3x3)
get_base_yaw() -> float
```

AERIAL (free-flying multirotor / quadcopter):

```
takeoff(height=0.5) -> bool
    Climb to ~height m altitude and hold a stable upright hover.
move_to(x, y, z, tol=0.1) -> bool
    Fly to absolute world point (x, y, z) m, staying upright.
    NOTE: three SCALAR args (not a vector).
hover(secs=2.0) -> bool
    Station-keep at the current pose for secs seconds.
land() -> bool
get_base_pose() -> (np.ndarray xyz, np.ndarray R3x3)
    World position + orientation of the drone body.
```

HUMANOID (bipedal robot):

```
stand_balance(secs=3.0) -> bool
    Hold a stable upright standing pose for secs seconds.
squat(depth=0.15, secs=3.0) -> bool
    Lower the torso by depth m then return to standing, balanced.
get_torso_height() -> float
get_base_pose() -> (np.ndarray xyz, np.ndarray R3x3)
```

CONVENTIONS:

- All positions are world-frame meters.
- For grasping: APPROACH 5cm above the body, DESCEND to within 2cm, gripper_close, then LIFT. The robot's grasp_radius is ~8cm; closer is more reliable.
- On exception, your program exits without retry -- write defensively (try/except around motions you expect to fail).

You will be told which primitives are actually available for this robot. Use only those.